



Alma Mater Studiorum-Università di Bologna
Scuola di Ingegneria

Gestione dell'I/O in Java I/O di testo & tokenizzazione

Corso di Laurea in Ingegneria Informatica
Anno accademico 2023/2024

Prof. ENRICO DENTI

Dipartimento di Informatica – Scienza e Ingegneria (DISI)

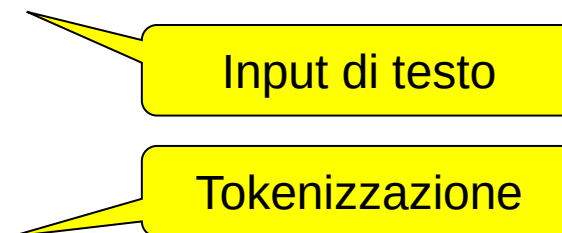


UN PO' DI STORIA

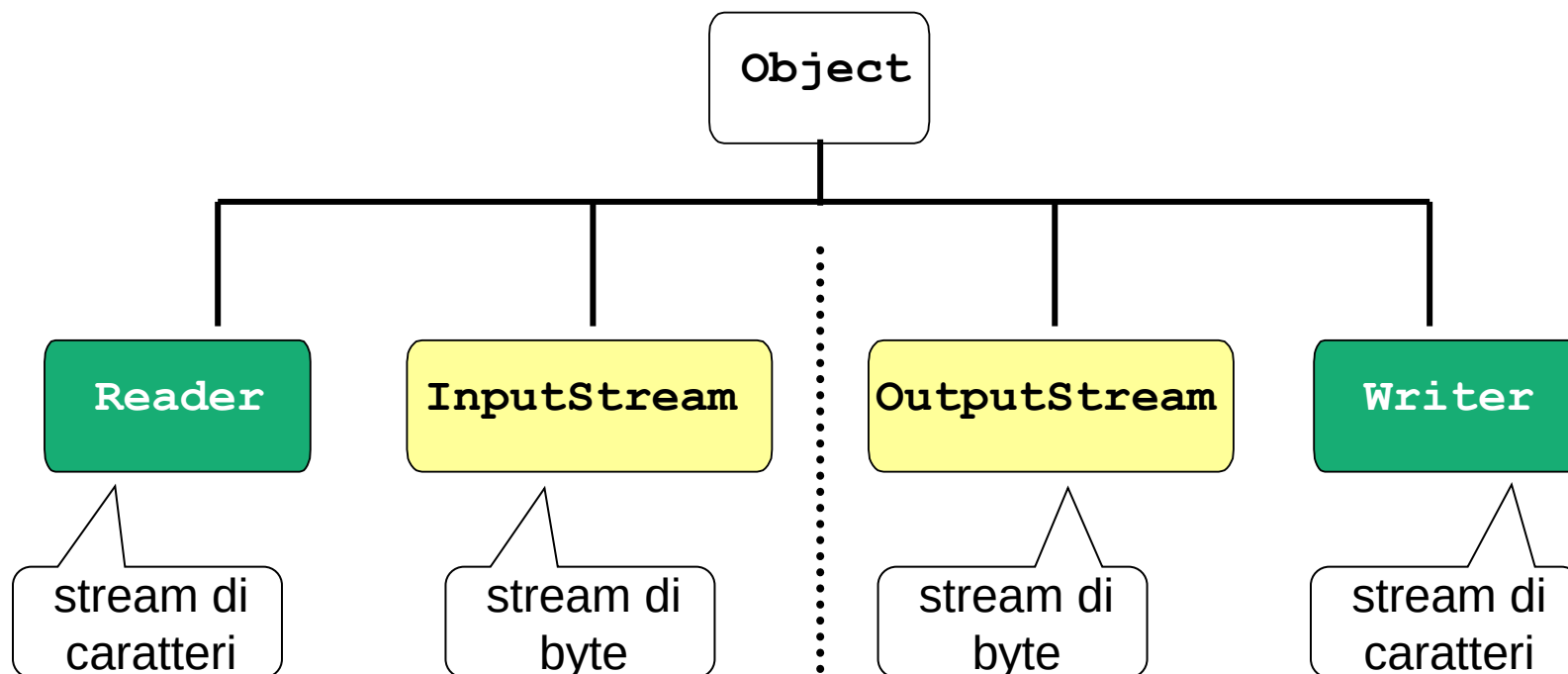
- Java 2
 - package fondamentale **java.io** e astrazioni correlate
- Java 6
 - nuova astrazione *console*
 - astrae dai molti dettagli di **System.in** e **System.out**
- Java 7
 - nuovo package **java.nio.file**
 - gestione completa dell'I/O da file (incluso *zip file system*)
 - nuova architettura
 - nuove classi factory & libreria (**Files**, **Paths**)
 - nuovo entry point **Path** (al posti di **File**)
 - nuova sintassi per le eccezioni (*try-with-resources*)

PECULIARITÀ DELL'INPUT DI TESTO

- Raramente tutto si riduce solo a leggere "del testo"
- Tipicamente, nelle applicazioni:
 - prima si **leggono** *interi righe di testo che contengono dati strutturati*
 - un treno con le sue fermate e orari
 - una valuta coi suoi tassi di cambio
 - ...
 - poi si **analizzano** le righe lette per *estrarre tali dati*
 - ad esempio, separare le singole fermate, i singoli orari, i tassi di cambio
 - infine si **usano** tali dati per *costruire oggetti (il modello dei dati)*
 - ad esempio, creare istanze di **Train** a partire dai dati letti
- Il package di I/O si occupa del *solo input*:
per la tokenizzazione si utilizzano classi accessorie



CLASSI BASE ASTRATTE



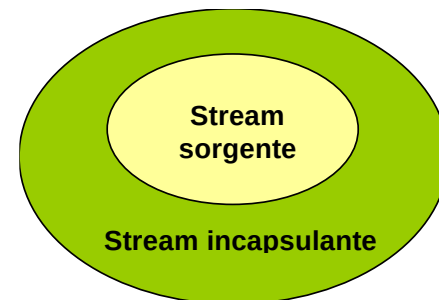
Reader e Writer

- sono *più efficienti* nella gestione di caratteri rispetto agli stream di byte
- *convertono correttamente* UNICODE nella codifica della piattaforma locale e tenendo conto della cultura locale (internazionalizzazione)

APPROCCIO "A CIPOLLA"

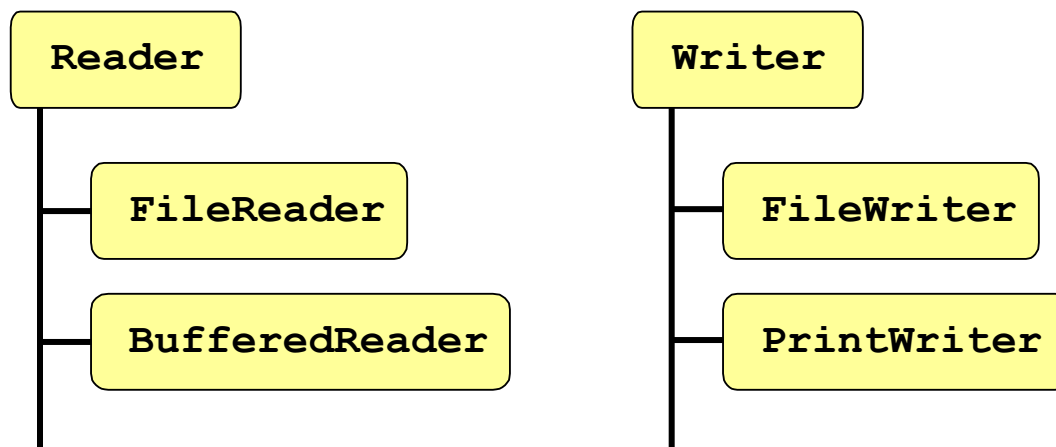
Anche gli stream di caratteri adottano l'approccio ad adapter

- al centro, stream che incapsulano *sorgenti fisiche* di dati o *dispositivi fisici di uscita*
 - i loro costruttori hanno come argomento *il dispositivo che interessa*: file, connessioni di rete, array di byte,...
- intorno, *stream di adattamento* che aggiungono funzionalità
 - i loro costruttori hanno come argomento *uno stream già esistente* (il nucleo da "avvolgere")



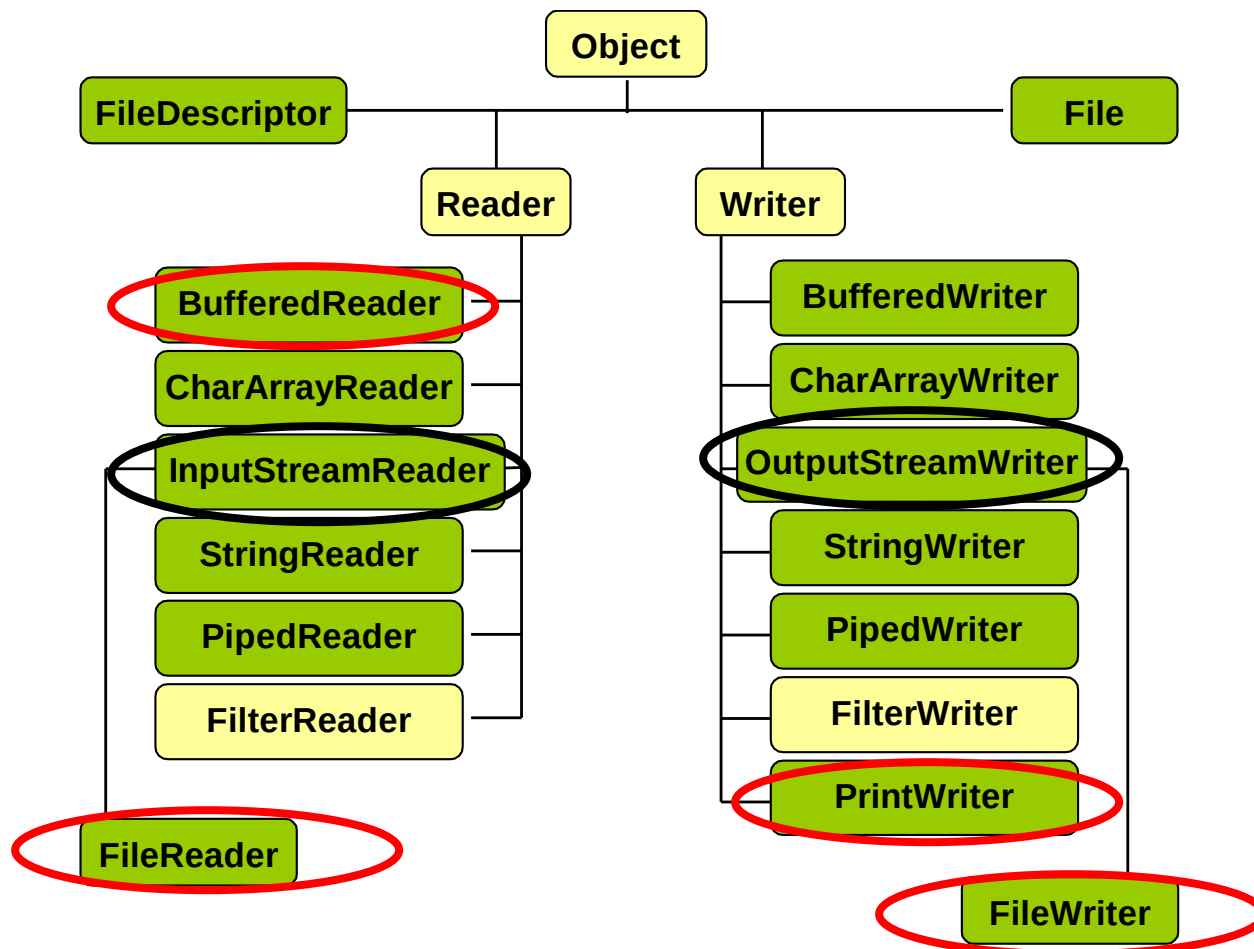
CLASSI CONCRETE

- Dalle due classi base derivano varie *classi concrete*
 - nucleo base: **FileReader** e **FileWriter**
 - adapter principali: **BufferedReader** e **PrintWriter**
 - altri adapter (verso stream binari, verso stringhe, etc.)
 - ...





TASSONOMIA COMPLETA





I/O DI TESTO

- Il file si apre istanziando un **FileReader** o **FileWriter**
- Il metodo **read** o **write** legge/scrive un **int** che contiene un carattere Unicode
 - ricorda: un carattere Unicode è lungo *due* byte
 - il metodo **read** restituisce -1 in caso di fine stream
 - occorre un *cast* per convertire il carattere Unicode letto in **int**
- Nei fatti, **FileReader** e **FileWriter** sono tipicamente incapsulati in un **BufferedReader** e in un **PrintWriter** (più raramente, nel secondo caso, in un **BufferedWriter**)
- La struttura generale del codice è *del tutto analoga* a quella vista per gli stream di byte.



ESEMPIO: INPUT DA FILE (1/3)

```
import java.io.*;
public class LetturaDaFileDiTesto {
    public static void main(String args[]){
        FileReader r = null;
        try {
            r = new FileReader(args[0]);
        }
        catch(FileNotFoundException e){
            System.out.println("File non trovato");
            System.exit(1);
        }
        // ... segue la lettura ...
    }
}
```



ESEMPIO: INPUT DA FILE (2/3)

```
...  
try {  
    int n=0, x = r.read();  
    while (x>=0) {  
        char ch = (char) x;  
        System.out.print(" " + ch); n++;  
        x = r.read();  
    }  
    System.out.println("\nTotale caratteri: " + n);  
} catch (IOException ex) {  
    System.out.println("Errore di input");  
    System.exit(2);  
}  
...
```

Cast esplicito da **int** a **char**
ma **solo** se è stato davvero letto un
carattere (cioè **x** non vale -1)



ESEMPIO: INPUT DA FILE (3/3)

Esempio d'uso:

```
C:\temp>java LetturaDaFileDiTesto prova.txt
```

Il risultato:

```
N e l   m e z z o   d e l   c a m m i n   d i  
n o s t r a   v i t a  
Totale caratteri: 35
```

Analogo esercizio può essere svolto per la scrittura su file di testo carattere per carattere.



INPUT BUFFERIZZATO

- **FileReader** legge solo *carattere per carattere*
- **BufferedReader** *incapsula un Reader* e lo dota di un buffer di accumulo, permettendo la *lettura per righe*
- Il metodo **readLine** restituisce la riga letta come **String**
 - se il file è finito, restituisce **null**
 - non lancia mai **EOFException** 😊
- La riga così acquisita può poi essere tokenizzata
 - a mano (giammai!)
 - tramite classi accessorie come **StringTokenizer** o **Scanner**
 - sfruttando il metodo **split** della classe **String**



ESEMPIO: INPUT BUFFERIZZATO

```
BufferedReader f = null;  
try {  
    f = new BufferedReader(new FileReader("f.txt"));  
    String riga;  
    while ((riga = f.readLine()) != null) {  
        ... // elaborazione riga  
    }  
}
```

Test sul fine file: quando il file è finito, **readLine** restituisce **null**

A ogni iterazione:

- si legge con **readLine** una nuova *potenziale* riga
- se il risultato non è **null**, la riga esiste → la si elabora



OUTPUT BUFFERIZZATO

- **FileWriter** scrive solo *carattere per carattere*
- **BufferedWriter** *incapsula un Writer* e lo dota di un buffer di accumulo, introducendo così la *nozione di riga*
 - il metodo **newline** emette un *separatore di riga*
 - offre una serie di metodi **write** per scrivere caratteri o stringhe
 - MA non offre metodi per una vera *stampa di righe*
- **PrintWriter** introduce la *vera stampa di righe*
 - i metodi **print/println** stampano *stringhe, oggetti, valori*
 - *gestisce le eccezioni* → non richiede **try/catch** 😊

ESEMPIO: OUTPUT BUFFERIZZATO

```
PrintWriter f =  
    new PrintWriter(new FileWriter("f.txt"));  
f.print(...);  
...  
f.println(...);
```

Trattandosi di una situazione *molto frequente*, esiste un costruttore che salta un passaggio, accettando direttamente il nome del file:

```
PrintWriter f =  
    new PrintWriter("f.txt");  
f.print(...);  
...  
f.println(...);
```

Costruttore diretto dal
nome del file



LA CLASSE `Console` DI JAVA 6

- La classe **`Console`** rappresenta la *console di sistema*
 - *può non esistere* se la JVM non è stata attivata "da console" (se è stata attivata da `javaw` o Eclipse...)
 - *se esiste, ce n'è un'unica istanza (singleton)*
 - per questo, non può essere costruita direttamente
 - si ottiene solo tramite il *metodo factory* **`System.console`**
- Si usa tramite il **`Reader`** e **`Writer`** associati
 - tali stream si recuperano tramite i due metodi `reader` e `writer`
- Cosa si può fare?
 - *leggere righe intere* tramite **`readLine`**
 - *scrivere righe formattate* con **`format`** e/o **`printf`** (ancora lei? ebbene sì! 😊)



Console: METODI

Method Summary

void	flush() Flushes the console and forces any buffered output to be written immediately .
Console	format(String fmt, Object... args) Writes a formatted string to this console's output stream using the specified format string and arguments.
Console	printf(String format, Object... args) A convenience method to write a formatted string to this console's output stream using the specified format string and arguments.
Reader	reader() Retrieves the unique Reader object associated with this console.
String	readLine() Reads a single line of text from the console.
String	readLine(String fmt, Object... args) Provides a formatted prompt, then reads a single line of text from the console.
char[]	readPassword() Reads a password or passphrase from the console with echoing disabled.
char[]	readPassword(String fmt, Object... args) Provides a formatted prompt, then reads a password or passphrase from the console with echoing disabled.
PrintWriter	writer() Retrieves the unique PrintWriter object associated with this console.



Console: CAUTELE D'USO

PRECISAZIONI IMPORTANTI SUI METODI

1. Invoking `close()` on the objects returned by `reader()` and `writer()` will **not** close the underlying stream of those objects.
2. The console-read methods *return null when the end of the console input stream is reached*, for example by typing control-D on Unix or control-Z on Windows. *Subsequent read operations will succeed if additional characters are later entered on the console's input device.*

RIGUARDO ALLA LETTURA DELLE PASSWORD:

1. To read a password or other secure data, an application should use `readPassword()` or `readPassword(String, Object...)`
2. After reading, it should *manually zero the returned character array* after processing, to minimize the lifetime of sensitive data in memory.



Console – ESEMPIO

```
Console console = System.console();  
String username = console.readLine("User Name? ");  
char[] password = console.readPassword("Password? ");  
...  
Arrays.fill(password, ' ');  
console.printf("Welcome, %1$s.", username);  
console.printf(System.getProperty("line.separator"));
```

Potrebbe essere *null*...

Cancellazione password

ESEMPIO DI RECUPERO PASSWORD & SUCCESSIVA CANCELLAZIONE

```
Console cons;  
char[] passwd;  
if ((cons = System.console()) != null &&  
    (passwd = cons.readPassword("[%s]", "Password:")) != null) {  
    ...  
    java.util.Arrays.fill(passwd, ' ');  
}
```

Console – ESEMPIO

```
Console console = System.console();
```

Potrebbe essere *null*...

```
ConsoleTest.java X
import java.io.Console;

public class ConsoleTest {
    public static void main(String[] args){
        Console console = System.console();
        if (console==null) {
            System.out.println("console is null");
            System.exit(1);
        }
        //else
        String username = console.readLine("User Name? ");
        char[] password = console.readPassword("Password? ");
        Arrays.fill(password, ' ');
        console.printf("Welcome, %1$s.", username);
        console.printf(System.getProperty("line.separator"));
    }
}
```

```
        System.out.println("User Name? ");
        console.readPassword("Password? ");
        Arrays.fill(password, ' ');
        console.printf("Welcome, %1$s.", username);
        console.printf(System.getProperty("line.separator"));
    }
}
```

...mentre qui
va tutto bene

Console X

```
<terminated> ConsoleTest [J... Java\jdk1.7.0_...
console is null
```

..E INFATTI...

C:\Windows\system32\cmd.exe

```
User Name? enrico
Password?
Welcome, enrico.
Premere un tasto per continuare . . .
```

Tokenizzazione di righe



DOPO LA LETTURA

- Una volta letta una riga, occorre **estrarre da ogni singola riga le parti costitutive, dette *token***
 - sottostringhe delimitate da separatori (caso tipico) o da una implicita informazione sulla loro lunghezza
 - si tratta solitamente di nomi, valori numerici, date, orari, etc.
- Occorre assolutamente evitare il «fai da te» manuale!
- Molto meglio **affidarsi a classi e funzionalità apposite:**
 - classe `java.util.StringTokenizer` (più antica)
 - classe `java.util.Scanner` (più moderna ed efficiente)
 - metodo ***split*** della classe `String`



StringTokenizer, Scanner, split

- Analogie e differenze: provenienza dell'input
 - **StringTokenizer** e **split** agiscono su una **singola riga** (già letta)
 - **Scanner** può agire in **più modi**, anche incapsulando un **Reader**
- Analogie e differenze: configurabilità
 - **StringTokenizer** è **semplice** ma **poco configurabile**
 - **Scanner** è **molto più sofisticata** (e quindi meno ovvia da usare) ma in grado di catturare moltissime situazioni → grande espressività
 - **split** è **una via di mezzo** pensata per catturare con efficacia e semplicità un caso d'uso specifico ma molto comune. Mantiene la potenza di **Scanner** relativamente all'espressività dei separatori, ma non ha la stessa generalità applicativa.



STRATEGIA DI FONDO

- Per estrarre i token, occorre in primis identificare **cosa li caratterizzi** e, dualmente, quali siano invece i **separatori**
- Domande chiave:
 - Tutti i token della riga sono separati dallo **stesso separatore**?
 - Se ci sono **separatori diversi**, è possibile trattarli come fossero un tutt'uno, o vanno invece trattati distintamente?
 - Può accadere che un separatore fra due token sia presente anche, come testo normale, in altri token?
- Le risposte a queste domande determinano le **strategie** da usare per una tokenizzazione efficace.

ALCUNI CASI TIPICI (1/2)

Esempio 1: *separatore fisso (\$)*

Giovanni Rossi \$ via Indipendenza 38\$Bologna
Gian Paolo Prinzi \$ via Altabella 46\$Bologna
Anna Maria Senzi\$ via dell'Arco 18/2 \$S.Lazzaro

Esempio 2: *separatori diversi, ma potenzialmente assimilabili*

Giovanni Rossi, via Indipendenza 38 - Bologna
Gian Paolo Prinzi, viale dei Mille 42 - Reggio Emilia
Anna Maria Senzi, via dell'Arco 18/2- S.Lazzaro

Esempio 3: *separatori diversi, da trattare distintamente*

Giovanni Rossi, via Indipendenza, 38 - Bologna
Gian Paolo Prinzi, viale dei Mille, 42 - Reggio Emilia
Anna Maria Senzi, via dell'Arco, 18/2- S.Lazzaro



ALCUNI CASI TIPICI (2/2)

Esempio 4: *separatori impliciti*

alfabetico *alfabetico* *numerico*

Madre Arianna3471234567

Padre Giovanni3349876543

Esempio 5: *larghezza fissa (non servono separatori)*

6 char *8 char* *10 char*

Madre Arianna 3471234567

FiglioGiovanni3349876543



StringTokenizer

- Il tokenizzatore **StringTokenizer** **incapsula una stringa già letta**: il metodo *nextToken* ne estrae via via i pezzi

```
var stk = new StringTokenizer(line) ;
```

- DIFETTO: la stringa incapsulata *non si può cambiare*
Ergo, per elaborare più righe occorre creare *un nuovo tokenizer per ogni riga*, generando decine di oggetti ☹

- Però, è semplice da usare e da configurare:

- il metodo *nextToken* estrae un token:

```
String token = stk.nextToken() ;
```

- il metodo *hasMoreTokens* dice se ci sono altri token:

```
while (stk.hasMoreTokens()) { ... }
```



StringTokenizer

- Per **StringTokenizer** un *token* è una sequenza di caratteri *fino al prossimo separatore (o a fine riga)*
- I separatori possono essere specificati:
 - all'inizio, alla costruzione del tokenizer

```
var stk = new StringTokenizer(line, ",");
```
 - successivamente, alla chiamata del metodo **nextToken**

```
String token = stk.nextToken(",");
```
- I separatori indicati rimangono *validi fino a successiva eventuale modifica*
 - si applicano quindi a *tutte* le chiamate al metodo **nextToken** fino a eventuale nuova diversa specifica



ESEMPIO 1

// FORMATO RIGA: "elemento,elemento,elemento"

```
try {  
    String line;  
    FileReader f = null;  
    // ... segue normale apertura del file  
    BufferedReader rdr = new BufferedReader(f);  
    while ((line=rdr.readLine()) != null) {  
        var stk = new StringTokenizer(line, ",");  
        String first  = stk.nextToken();  
        String second = stk.nextToken();  
        String third  = stk.nextToken();  
        // fai quel che devi con first, second, third  
    }  
    ...  
}
```



ESEMPIO 1

// FORMATO RIGA: "elemento, elemento, elemento "

```
try {  
    String line;  
    FileReader f = null;  
    // ... segue normale apertura del file  
    BufferedReader rdr = new BufferedRead  
    while ((line=rdr.readLine()) != null  
        var stk = new StringTokenizer(line,  
        String first  = stk.nextToken().trim();  
        String second = stk.nextToken().trim();  
        String third  = stk.nextToken().trim();  
        // fai quel che devi con first, second, third  
    }  
    ...
```

Utile `trim` per eliminare
eventuali spazi spuri
davanti/dietro ai token

ESEMPIO 1 – variante

- È possibile specificare come possibili separatori *anche più caratteri*: il tokenizer considererà come *singolo separatore* una *qualsiasi sequenza* di tali caratteri, *comunque mischiati*
 - ESEMPIO: possibili separatori «virgola» e «punto e virgola»
`var stk = new StringTokenizer(line, ",;");`
 - Sono validi separatori «*;*», «*;;*», «*,,*», «*,;,;*», etc.
 - Ergo, le seguenti righe sono tutte tokenizzate allo stesso modo:
`Giovanni,Rossi;Bologna`
`Giovanni;Rossi,Bologna`
`Giovanni;;;Rossi,;Bologna`
 - NB: l'ultimo token non ha separatore esplicito, ma *il fine riga è sempre implicitamente un separatore valido*

ESEMPIO 1 – variante

```
import java.util.StringTokenizer;
import java.io.*;

class HelloWorld {
    public static void main(String[] args) throws IOException {
        var fakeFile = "Giovanni,Rossi,Bologna\n" +
                       "Giovanni,Rossi,Bologna\n" +
                       "Giovanni;;;Rossi,;Bologna";
        var reader = new BufferedReader(new StringReader(fakeFile));

        String line;
        while((line=reader.readLine())!=null) {
            var stk = new StringTokenizer(line, ",;");
            String first = stk.nextToken().trim();
            String second = stk.nextToken().trim();
            String third = stk.nextToken().trim();
            System.out.println(first + "/" + second + "/" + third);
        }
    }
}
```

Lo **StringReader** genera un reader alimentato da una stringa: utile per test

Una qualsiasi sequenza di «virgola» e/o «punto e virgola» costituisce un singolo separatore

```
java -cp /tmp/WE1XaiEDRQ/HelloWorld
Giovanni/Rossi/Bologna
Giovanni/Rossi/Bologna
Giovanni/Rossi/Bologna
```

```
=== Code Execution Successful ===
```

ESEMPIO 2

- Può accadere però che il separatore, all'interno di una riga, *non sia sempre lo stesso*, ad esempio:

Madre: Arianna, Bianchi

Padre: Giovanni, Rossi

Figlio: Gian Marco, Rossi

- **DOMANDA CHIAVE:** quei separatori sono assimilabili a un unico separatore o vanno trattati diversamente?
 - ossia: sarebbero accettabili righe come le seguenti?
Madre, Arianna: Bianchi
Padre: :Giovanni, :Rossi
 - se sì, si ricade nel caso dell'Esempio 1
 - se no, occorre un *trattamento ad hoc*

ESEMPIO 2

- IPOTESI: i separatori sono diversi nel senso che *è certamente il «:» a separare i primi due e la «,» i secondi due*
 - ciò equivale a dire che *non è accettabile* considerare i due separatori alla stregua di un unico separatore «esteticamente variegato»
- CONSEGUEENZA: il tokenizer deve usare *solo il «:» nella prima parte della riga e solo la «,» nella seconda parte*
 - occorre quindi suddividere il lavoro in DUE FASI, *cambiando il delimitatore* dopo aver estratto il primo token
 - MA ATTENZIONE: così facendo, *il precedente separatore torna ad essere un carattere «normale»* e come tale sarà trattato nella successiva tokenizzazione
 - rischio di ritrovarselo «in mezzo» nel successivo token

ESEMPIO 2 (codice imperfetto)

```
import java.util.StringTokenizer;
import java.io.*;

class HelloWorld {
    public static void main(String[] args) throws IOException {
        var fakeFile = "Madre: Arianna,Bianchi\n" +
            "Padre: Giovanni,Rossi\n" +
            "Figlio: Gian Marco, Rossi";
        var reader = new BufferedReader(new StringReader(fakeFile));

        String line;
        while((line=reader.readLine())!=null) {
            var stk = new StringTokenizer(line, ":");
            String first = stk.nextToken().trim();
            String second = stk.nextToken(",").trim();
            String third = stk.nextToken().trim();
            System.out.println(first + "/" + second + "/" + third);
        }
    }
}
```

Il primo token termina al
«due punti» 😊

Il secondo token termina alla
«virgola», **MA** partendo dal
precedente «due punti» 😞

Il terzo token termina alla
«virgola» o a fine riga 😊

```
java -cp /tmp/5QPLw1WwHJ/HelloWorld
Madre/: Arianna/Bianchi
Padre/: Giovanni/Rossi
Figlio/: Gian Marco/Rossi

=== Code Execution Successful ===
```

ESEMPIO 2 – variante

- Aver cambiato il separatore «al volo» ha fatto sì che il precedente separatore sia stato incluso nel secondo token
- Per evitarlo occorre *distinguere i due parsing in due momenti distinti, svolti successivamente* → servono due tokenizer!
 - questo genera purtroppo ancora più garbage...

Queste situazioni si gestiranno meglio con Scanner o split

SCHEMA

```
var stk = new StringTokenizer(line, ":" );
String first  = stk.nextToken().trim();
String others = stk.nextToken().trim();

var stk2 = new StringTokenizer(others, "," );
String second = stk2.nextToken().trim();
String third  = stk2.nextToken().trim();
```

ESEMPIO 2 – variante

```
import java.util.StringTokenizer;
import java.io.*;

class HelloWorld {
    public static void main(String[] args) throws IOException {
        var fakeFile = "Madre: Arianna,Bianchi\n" +
                       "Padre: Giovanni,Rossi\n" +
                       "Figlio: Gian Marco, Rossi";
        var reader = new BufferedReader(new StringReader(fakeFile));

        String line;
        while((line=reader.readLine())!=null) {
            var stk = new StringTokenizer(line, ":");
            String first = stk.nextToken().trim();
            String others = stk.nextToken().trim();
            var stk2 = new StringTokenizer(others, ",");
            String second = stk2.nextToken().trim();
            String third = stk2.nextToken().trim();
            System.out.println(first + "/" + second + "/" + third);
        }
    }
}
```

Il primo token termina al «due punti»; il secondo token incorpora *il resto della riga*

Tale «resto» viene incapsulato in altro tokenizer, che lo separa usando la «virgola»

```
java -cp /tmp/M43nZE2SJr/HelloWorld
Madre/Arianna/Bianchi
Padre/Giovanni/Rossi
Figlio/Gian Marco/Rossi
```

```
=== Code Execution Successful ===
```

ESEMPIO 3

- Finora, il separatore è sempre stato *esplicito*
- A volte però non c'è un separatore esplicito: **la demarcazione fra due token consecutivi può essere *implicita***, se due token adiacenti contengono caratteri *mutuamente esclusivi*
 - situazione tipica: primo token alfabetico, secondo token numerico
 - ESEMPIO: una riga del tipo ***parola342423423***
- Questo tipo di situazione è più facile da gestire con **`StringTokenizer`** che con **`Scanner`** o **`split`**
 - inizialmente, si impostano come separatori i caratteri numerici
→ si legge fino al «3» escluso
 - poi, si cambiano mettendo il fine riga
→ si legge la parte numerica fino alla fine

ESEMPIO 3

```
// FORMATO RIGA: "nome parola342423423"
```

```
try {  
    String line;  
    FileReader f = null;  
    // ... segue normale apertura del file  
    BufferedReader rdr = new BufferedReader(f);  
    while ((line=rdr.readLine()) != null) {  
        StringTokenizer stk = new StringTokenizer(line);  
        String first  = stk.nextToken().trim();  
        String second = stk.nextToken("0123456789").trim();  
        String third  = stk.nextToken("\n\r").trim();  
        // fai quel che devi con first, second, third  
    }  
    ...  
}
```

Primo token:
fino a spazi, tab

Secondo token: fino
a cifre numeriche

Terzo token:
fino a fine riga

ESEMPIO 3

```
import java.util.StringTokenizer;
import java.io.*;

class HelloWorld {
    public static void main(String[] args) throws IOException {
        var fakeFile = "Madre  Arianna3471234567\n" +
            "Padre  Giovanni3341234567\n" +
            "Figlio Gian Marco3891234567";
        var reader = new BufferedReader(new StringReader(fakeFile));

        String line;
        while((line=reader.readLine())!=null) {
            var stk = new StringTokenizer(line);
            String first = stk.nextToken().trim();
            String second = stk.nextToken("0123456789").trim();
            String third = stk.nextToken("\n\r").trim();
            System.out.println(first + "/" + second + "/" + third);
        }
    }
}
```

Il primo token termina allo spazio o tab, il secondo va *fino al primo carattere numerico*

Il terzo prosegue fino a fine riga

```
java -cp /tmp/Ff74qhXJA8/HelloWorld
Madre/Arianna/3471234567
Padre/Giovanni/3341234567
Figlio/Gian Marco/3891234567

=== Code Execution Successful ===
```



Scanner & split

- La classe **Scanner** ha l'obiettivo di:
 - superare la limitazione di **StringTokenizer** di lavorare su una stringa pre-incapsulata, così da evitare la creazione di N tokenizer
 - essere costruibile non solo da stringa, ma anche da **Reader**, **File** o **InputStream**, così da *gestire direttamente anche la tastiera*
 - gestire le problematiche dei diversi *character encoding* e tenere conto delle *culture locali* nella scansione di numeri ("." vs ",", " ")
- È **molto più potente di StringTokenizer** ma anche *più complessa da usare* al di fuori dei casi «banali»
- È usata internamente anche dal metodo **split** di **String** che *sfrutta la stessa notazione* per i suoi *delimitatori*



Scanner & split

- **Scanner** funziona in modo simile a **StringTokenizer**:
 - si costruisce e si configura il tokenizzatore con un certo *delimitatore*
 - si chiama ripetutamente un metodo **nextXXX**
 - si ottengono i vari token uno alla volta
- Il metodo **split** di **String** usa **Scanner** internamente:
utile quando il pattern di separazione è sempre lo stesso
 - si chiama il metodo una sola volta, passando il *delimitatore* da usare (lo stesso che si sarebbe dato a **Scanner**)
 - si ottengono tutti i token in un colpo solo, come *array* di stringhe
- L'uso dell'uno o dell'altro dipende dalle situazioni
- @ZioEnrico: **split** si adatta a molte casi pratici... 😊



Scanner – COSTRUZIONE

- Si può costruire uno scanner:

- da una *stringa* già nota e letta:

```
Scanner s1 = new Scanner(riga);
```

- da un *Reader* (più esattamente, da un *Readable*):

```
Scanner s2 = new Scanner(new SomeReader(...));
```

- da un *File*:

Questo costruttore può lanciare eccezione

```
Scanner s3 = new Scanner( new File("mynum.txt") );
```

- da *tastiera*:

```
Scanner kbd = new Scanner( System.in );
```

Costruttore da *InputStream*



Scanner – USO

- Una volta costruito, allo scanner si può chiedere:
 - se il prossimo token è ciò che ci si aspetta
 - `hasNext`, `hasNextInt`, `hasNextBoolean`, `hasNextFloat`, `hasNext(pattern)`, ...
 - (se sì), di *estrarre un token di un ben preciso tipo*:
 - `next`, `nextInt`, `nextBoolean`, `nextFloat`, `next(pattern)`, ...
 - verificare o *cambiare i delimitatori*
 - `delimiter`, `useDelimiter`, ...
- Il *delimitatore* e il *pattern* si basano sulla notazione delle *espressioni regolari*: potente, ma non sempre immediata

```
java.util.regex.Pattern
```



Scanner - ESEMPI BASICI

- CASI SENZA PATTERN (token separati da spazi e tab)

- lettura di un intero e di una parola da tastiera:

```
Scanner sc = new Scanner(System.in);  
int i = sc.nextInt(); String word = sc.next();
```

- lettura di una serie di interi long da file di testo:

```
Scanner sc = new Scanner(new File("numeri.txt"));  
while (sc.hasNextLong()) {  
    long x = sc.nextLong(); myList.add(x); }  
}
```

- cambio di delimitatori:

```
sc.useDelimiter(":");
```

- ritorno ai delimitatori predefiniti:

```
sc.reset();
```

ESEMPIO 4

- Se il separatore è sempre lo stesso, ottenere i token con **Scanner** è semplice: basta specificarlo come *delimitatore*

VERSIONE A: separatore «virgola»

```
Scanner sc = new Scanner(reader);  
sc.useDelimiter(",");  
while(sc.hasNext()) {  
    System.out.println(sc.next().trim());  
}
```

Pattern di separazione

VERSIONE B: separatore «una o più virgole»

```
Scanner sc = new Scanner(reader);  
sc.useDelimiter(",+");  
...
```

NEI PATTERN DI SEPARAZIONE:

+ significa 1+

* significa 0+

ESEMPIO 4

```
import java.util.Scanner;
import java.io.*;

class HelloWorld {
    public static void main(String[] args) throws IOException {
        var fakeFile = "Arianna, Bianchi, 3471234567\n" +
            "Giovanni, Rossi, 3341234567\n" +
            "Gian Marco, Rossi, 3891234567";
        var reader = new StringReader(fakeFile);

        Scanner sc = new Scanner(reader);
        sc.useDelimiter(",");
        while(sc.hasNext()) {
            System.out.println(sc.next().trim());
        }
    }
}
```

Il separatore è **una sola** virgola. Se una riga ne avesse 2+, lo scanner restituirebbe token intermedi vuoti.

Qui invece il separatore è **una o più** virgole: lo scanner restituisce token «puliti» anche in presenza di 2+ virgole.

```
import java.util.Scanner;
import java.io.*;

class HelloWorld {
    public static void main(String[] args) throws IOException {
        var fakeFile = "Arianna, Bianchi,, 3471234567\n" +
            "Giovanni, Rossi, 3341234567\n" +
            "Gian Marco, Rossi, 3891234567";
        var reader = new StringReader(fakeFile);

        Scanner sc = new Scanner(reader);
        sc.useDelimiter(",+");
        while(sc.hasNext()) {
            System.out.println(sc.next().trim());
        }
    }
}
```



Scanner – DELIMITATORI E PATTERN

- Alcune notazioni di uso comune
 - `\\s` uno spazio o assimilato (tab, etc.)
 - `\\t` una tabulazione
 - `\\d` una cifra numerica (`\\D` non cifra numerica)
 - `\\w` un carattere non bianco
 - Da combinare con:
 - `^...` negazione (il contrario di ciò che segue)
 - `...*` zero o più
 - `...+` uno o più
 - `(...)` raggruppamento
- | ESEMPI | |
|-------------------|---------------------------------|
| <code>\\s+</code> | uno o più spazi (e assimilati) |
| <code>\\t+</code> | uno o più tab |
| <code>\\d+</code> | uno o più cifre |
| <code>\\w+</code> | uno o più caratteri non bianchi |



RIPRENDENDO L'ESEMPIO 3

```
// FORMATO RIGA: "nome parola342423423"
```

```
try {
```

```
    FileReader f = null;
```

```
    // ... normale apertura del file
```

```
    // NB: non occorre più il BufferedReader!
```

```
    Scanner sc = new Scanner(f);
```

```
    while(sc.hasNext()) { {
```

```
        String first = sc.next().trim();
```

```
        sc.useDelimiter("\\d+");
```

```
        String second = sc.next().trim();
```

```
        sc.reset();
```

```
        String third = sc.next().trim();
```

```
        first, second, third
```

```
    }
```

```
    ...
```

Imposta come pattern
delimitatore *uno o più*
caratteri numerici

Torna al pattern
delimitatore di default

RIPRENDENDO L'ESEMPIO 3

```
import java.util.Scanner;
import java.io.*;

class HelloWorld {
    public static void main(String[] args) throws IOException {
        var fakeFile = "Madre  Arianna3471234567\n" +
            "Padre  Giovanni3341234567\n" +
            "Figlio Gian Marco3891234567";
        var reader = new StringReader(fakeFile);

        Scanner sc = new Scanner(reader);
        while(sc.hasNext()) {
            String first = sc.next().trim();
            sc.useDelimiter("\\d+");
            String second = sc.next().trim();
            sc.reset();
            String third = sc.next().trim();
            System.out.println(first + "/" + second + "/" + third);
        }
    }
}
```

Il primo token termina allo spazio o tab, il secondo va *fino al primo carattere numerico*

Il terzo prosegue fino a fine riga

```
java -cp /tmp/Ff74qhXJA8/HelloWorld
Madre/Arianna/3471234567
Padre/Giovanni/3341234567
Figlio/Gian Marco/3891234567
```

=== Code Execution Successful ===



Scanner - analisi di un'intera riga con pattern (difficile)

- In alternativa al ciclo e ai delimitatori, il metodo **findInline** consente di *specificare l'intero pattern della riga*
 - consuma e restituisce la parte di riga corrispondente al pattern
 - consuma il fine riga solo se nel pattern è previsto anche lo «\n»: se non è previsto, si può consumarlo invocando **nextLine()**
- **MA attenzione: specificarlo *bene* non è affatto semplice!!**
 - se tutto ok, la successiva chiamata al metodo **match** cattura gli elementi e li restituisce sotto forma di **MatchResult**, da cui possono essere estratti uno ad uno come stringa, tramite **group(index)**
 - l'indice in **group(index)** parte da 1 (l'elemento 0 è l'intera stringa)
- **@ZioEnrico: meglio evitare (se non siete *sicurissimi!*)**



RIPRENDENDO L'ESEMPIO 3 (approccio difficile)

- Il solito esempio svolto con `findInLine`

```
// FORMATO RIGA: "nome parola342423423"
```

```
Scanner sc = new Scanner(reader)
```

```
while(sc.hasNext()) {
```

```
    var r = sc.findInLine("(\\w+) (?:\\s+) (\\D+) (\\d+)");
```

```
    var res = sc.match();
```

```
    String first = res.group(1);
```

```
    String second = res.group(2);
```

```
    String third = res.group(3);
```

```
    System.out.println(first + "/" + second + "/" + third);
```

```
    if(sc.hasNextLine()) sc.nextLine();
```

```
}
```

1° token: Parola
(token ignorato: spazi)
2° token: Parola non numerica
3° token: numero



RIPRENDENDO L'ESEMPIO 3 (approccio difficile)

```
import java.util.Scanner;
import java.io.*;

class HelloWorld {
    public static void main(String[] args) throws IOException {
        var fakeFile = "Madre Arianna3471234567\n" +
            "Padre Giovanni3341234567\n" +
            "Figlio Gian Marco3891234567";
        var reader = new StringReader(fakeFile);

        Scanner sc = new Scanner(reader);
        while(sc.hasNext()){
            var r = sc.findInLine("(\\w+)(?:\\s+)(\\d+)(\\d+)");
            var res = sc.match();
            String first = res.group(1);
            String second = res.group(2);
            String third = res.group(3);
            System.out.println(first + "/" + second + "/" + third);
            if(sc.hasNextLine()) sc.nextLine();
        }
    }
}
```

1° token: Parola
(token ignorato: spazi)
2° token: Parola non numerica
3° token: numero

Consuma l'eventuale fine riga

```
java -cp /tmp/Ff74qhXJA8/HelloWorld
Madre/Arianna/3471234567
Padre/Giovanni/3341234567
Figlio/Gian Marco/3891234567
```

```
=== Code Execution Successful ===
```



String: split

- **split** è un metodo della classe **String** che sfrutta internamente uno **Scanner** per tokenizzare una stringa
 - l'argomento è il pattern da usare per separare i pezzi (quello che **Scanner** userebbe in **useDelimiter**)
 - facile da usare nei casi semplici (delimitatore sempre uguale), decisamente meno adatto e più complicato se il delimitatore cambia
- Ad esempio, per tokenizzare una riga con elementi separati da una singola «virgola», basta scrivere:

```
String[] parti = riga.split(",");
```

 - MA scrivendo così è necessario usare **trim** a tappeto per rimuovere gli spazi in eccesso davanti/dietro ai singoli token ☹



split – ESEMPIO

- Situazione tipica: stringa costituita da token separati da un singolo pattern costante, ad esempio una singola virgola:

```
String riga =  
"Giulio Verdi, via Indipendenza 38 , Bologna";
```

- Splittandola semplicisticamente con:

```
String[] parti = riga.split(",");
```

- si ottengono token corretti, MA con spazi spuri:

```
parti[0] = "Giulio Verdi"  
parti[1] = " via Indipendenza 38 "  
parti[2] = " Bologna"
```

Per liberarsi degli
spazi spuri, **trim** !



split – ESEMPIO MIGLIORATO

- MIGLIORIA: in alternativa alla `trim`, possiamo **migliorare la specifica del separatore** affinché *elimini lui gli spazi*

- Anziché:

```
String[] parti = riga.split(",");
```

- conviene scrivere:

```
String[] parti = riga.split("\\s*,\\s*");
```

- Così facendo, **gli spazi spuri sono automaticamente eliminati** perché fanno parte anch'essi del separatore, senza bisogno di `trim` aggiuntive 😊
 - se le virgole possono essere anche più di una, inserire `+` nel pattern
→ `"\\s*,+\\s*"`



split – ESEMPIO (VARIANTE)

- Se invece della «virgola» il separatore fosse consistito in una o più tabulazioni (caso più realistico):

```
String riga =  
"Giulio Verdi\tvia Indipendenza 38\t\tBologna";
```

- Splittandola con:

```
String[] parti = riga.split("\\s*\\t+\\s*");
```

- si ottiene:

```
parti[0] = "Giulio Verdi"  
parti[1] = "via Indipendenza 38"  
parti[2] = "Bologna"
```

Essenziale il + per
catturare anche *più*
tabulazioni consecutive

split – ESEMPIO (VARIANTE)

```
class HelloWorld {  
    public static void main(String[] args) {  
        var riga = "Giulio Verdi\t via Indipendenza 38 \t\t Bologna";  
        String[] parti = riga.split("\\s*\\t+\\s*");  
        System.out.println(String.join("/",parti));  
    }  
}
```

```
java -cp /tmp/SjlDyuNY29/HelloWorld  
Giulio Verdi/via Indipendenza 38/Bologna
```

```
=== Code Execution Successful ===
```

Il **delimitatore furbo** «tab preceduto e/o seguito da spazi» elimina da solo gli spazi spuri 😊

```
class HelloWorld {  
    public static void main(String[] args) {  
        var riga = "Giulio Verdi\t via Indipendenza 38 \t\t Bologna";  
        String[] parti = riga.split("\\t+");  
        System.out.println(String.join("/",parti));  
    }  
}
```

```
java -cp /tmp/gWZDpcJa4Q/HelloWorld  
Giulio Verdi/ via Indipendenza 38 / Bologna
```

```
=== Code Execution Successful ===
```

Col **delimitatore basico** (solo tab) avremmo avuto molti spazi spuri 😞



split – RIASSUNTO

- `\\s` significa **spazi e assimilati**, quindi *anche tabulazioni* e ogni altro carattere «whitespace-like»
- `\\t` significa invece **solo tabulazioni**
- Quindi, una specifica come la seguente:
`String[] parti = riga.split("\\s+");`
- taglia la riga **a ogni sequenza** di **spazi e/o tabulazioni**

```
class HelloWorld {  
    public static void main(String[] args) {  
        var riga = "Giulio Verdi\\t via Indipendenza 38 Bologna";  
        String[] parti = riga.split("\\s+");  
        System.out.println(String.join("/", parti));  
    }  
}
```

```
java -cp /tmp/SXu070iCha/HelloWorld  
Giulio/Verdi/via/Indipendenza/38/Bologna  
  
=== Code Execution Successful ===
```



UN CASO MOLTO COMPLICATO

IL FAMIGERATO ESEMPIO 3

// *FORMATO RIGA: "nome parola342423423"*

- L'esempio 3 non si può gestire direttamente con *split* perché il pattern esprime *tutti* i possibili separatori *insieme*
 - non concepisce che alcuni possano essere usati «solo prima» e altri «solo dopo» nella medesima riga!
- Per fare ciò occorrono *più split in cascata*, separando la stringa in più tempi, ognuna col proprio separatore
 - nel caso dell'Esempio 3, ciò che complica tutto è *l'assenza di un separatore esplicito fra secondo e terzo elemento*
 - occorrerà fare i «salti mortali» usando due split con separatori «opposti» per isolare, rispettivamente, solo il secondo elemento o solo il terzo
 - contorto, error prone + probabile comparsa di token vuoti spuri



UN CASO MOLTO COMPLICATO

IL FAMIGERATO ESEMPIO 3

// FORMATO RIGA: "nome parola342423423"

```
try {
```

Decisamente, qui NON è l'approccio giusto ☹

```
    FileReader f = null;
```

```
    // ... segue normale apertura del file
```

```
    BufferedReader rdr = new BufferedReader(f);
```

```
    while((line=f.readLine())!=null) {
```

```
        String[] itemsLevel1 = line.split("\\s+");
```

```
        String[] itemsLevel2 =
```

```
            itemsLevel1[1].trim().split("\\d+");
```

Separatori=cifre

```
        String[] itemsLevel3 =
```

```
            itemsLevel1[1].trim().split("\\D+");
```

Separatori=non cifre

```
        // fai quel che devi con first = itemsLevel1[0]
```

```
        // fai quel che devi con second = itemsLevel2[0]
```

```
        // fai quel che devi con third = itemsLevel3[1]
```

```
    ...
```

```
}
```

Disuniformità: ci viene in mezzo un token vuoto ☹

OCCHIO all'ultimo indice

UN CASO MOLTO COMPLICATO

IL FAMIGERATO ESEMPIO 3

- A peggiorare le cose, il test case con «Gian Marco» fallisce perché contiene anch'esso uno spazio

```
import java.io.*;

class HelloWorld {
    public static void main(String[] args) throws IOException {
        var fakeFile = "Madre  Arianna3471234567\n" +
                       "Padre  Giovanni3341234567\n" +
                       "Figlio Gian Marco3891234567";

        var reader = new BufferedReader(new StringReader(fakeFile));

        String line;
        while((line=reader.readLine())!=null) {
            var items = line.split("\\s+");
            String first = items[0].trim();
            String others = items[1].trim();
            items = others.split("\\d+");
            String second = items[0].trim();
            items = others.split("\\D+");
            String third = items[1].trim();
            System.out.println(first + "/" + second + "/" + third);
        }
    }
}
```

Fallisce tutto perché in effetti non è una singola parola!

Disuniformità: l'ultimo item ha indice 1 perché prima ci viene pure un elemento vuoto "" !

```
import java.io.*;

class HelloWorld {
    public static void main(String[] args) throws IOException {
        var fakeFile = "Madre  Arianna3471234567\n" +
                       "Padre  Giovanni3341234567\n" +
                       "Figlio GianMarco3891234567";

        var reader = new BufferedReader(new StringReader(fakeFile));

        String line;
        while((line=reader.readLine())!=null) {
            var items = line.split("\\s+");
            String first = items[0].trim();
            String others = items[1].trim();
            items = others.split("\\d+");
            String second = items[0].trim();
            String third = items[1].trim();
            System.out.println(first + "/" + second + "/" + third);
        }
    }
}
```

Così invece funzionerebbe



splitWithDelimiters

UN NUOVO METODO in Java 21

- Da **Java 21** il nuovo metodo **splitWithDelimiters** consente di gestire meglio casi come questi
- Questo metodo, a differenza di **split**, **restituisce nell'array items anche le occorrenze del delimitatore**
 - ☺ così, nel caso dell'esempio 3, l'ultimo elemento (third) viene fuori «gratis» come sottoprodotto del secondo
 - ☺ però, si generano token «vuoti» (sono anch'essi delimitatori...)
- Nell'esempio 3, il delimitatore «spazi o cifre» "**\\s+|\\d+**" produce (quasi) quanto richiesto:
 - riga: «**nome** **parola342423423**»
 - token: «**nome**», «», «**parola**», «**342423423**»
 Token extra, spurio



IL FAMIGERATO ESEMPIO 3

COL NUOVO METODO DI Java 21

- NB: il metodo prevede un secondo argomento numerico, dalla particolare semantica: tipicamente va lasciato a 0

```
BufferedReader rdr = new BufferedReader(f);  
while ((line=rdr.readLine()) != null) {  
    String[] items = line.splitWithDelimiters(  
        "\\s+|\\d+", 0);  
  
    String first  = items[0].trim();  
    String second = items[2].trim();  
    String third  = items[3].trim();  
  
    System.out.println(first + "/" + second + "/" + third);  
}
```

Il token #1 (spazi)
non ci interessa

NB: come prima, funziona bene solo se non ci sono spazi nei nomi (altrimenti, usare TAB come separatore)

```
Madre/Arianna/3471234567  
Padre/Giovanni/3341234567  
Figlio/GianMarco/3891234567
```

Approcci tipici per casi concreti

ALCUNI CASI TIPICI (1/2)

Esempio 1: *separatore fisso (\$)*

Giovanni Rossi \$ via Indipendenza 38\$Bologna
Gian Paolo Prinzi \$ via Altabella 46\$Bologna
Anna Maria Senzi\$ via dell'Arco 18/2 \$S.Lazzaro

Esempio 2: *separatori diversi, ma potenzialmente assimilabili*

Giovanni Rossi, via Indipendenza 38 - Bologna
Gian Paolo Prinzi, viale dei Mille 42 - Reggio Emilia
Anna Maria Senzi, via dell'Arco 18/2- S.Lazzaro

Esempio 3: *separatori diversi, da trattare distintamente*

Giovanni Rossi, via Indipendenza, 38 - Bologna
Gian Paolo Prinzi, viale dei Mille, 42 - Reggio Emilia
Anna Maria Senzi, via dell'Arco, 18/2- S.Lazzaro



ALCUNI CASI TIPICI (2/2)

Esempio 4: *separatori impliciti*

alfabetico *alfabetico* *numerico*

Madre Arianna3471234567

Padre Giovanni3349876543

Esempio 5: *larghezza fissa (non servono separatori)*

6 char *8 char* *10 char*

Madre Arianna 3471234567

FiglioGiovanni3349876543

1° CASO

Esempio 1: *separatore fisso (\$)*

```
Giovanni Rossi $   via Indipendenza 38$Bologna  
Gian Paolo Prinzi $ via Altabella 46$Bologna  
Anna Maria Senzi$ via dell'Arco 18/2 $S.Lazzaro
```

- **StringTokenizer**: basta impostare il **\$** come delimitatore
 - MA i vari token dovranno essere puliti uno ad uno con **trim**
- **Scanner**: impostare come delimitatore il **\$** in **OR** col **carattere di fine riga \n**, per separare le righe fra loro
 - ATTENZIONE: **\$** è un carattere speciale, serve l'escape **\\\$**
 - si può evitare **trim** includendo nel pattern gli spazi **\\s*** intorno
- **split**: occorre impostare come delimitatore il **\$**
 - come con **Scanner**, essenziale l'escape **\\\$** e utile **\\s*** intorno



1° CASO – StringTokenizer

```
import java.util.Scanner;
import java.util.StringTokenizer;
import java.io.*;

class HelloWorld {
    public static void main(String[] args) throws IOException {
        var fakeFile = "Giovanni Rossi $   via Indipendenza 38$Bologna\n" +
            "Gian Paolo Prinzi  $ via Altabella 46$Bologna\n" +
            "Anna Maria Senzi$ via dell'Arco 18/2 $S.Lazzaro";

        var reader = new BufferedReader(new StringReader(fakeFile));
        String riga;
        while((riga=reader.readLine())!=null){
            var stk = new StringTokenizer(riga, "$");
            var nomeCognome = stk.nextToken().trim();
            var indirizzo = stk.nextToken().trim();
            var citta = stk.nextToken().trim();
            System.out.println(nomeCognome + "|" + indirizzo + "|" + citta);
        }
    }
}
```

In luogo della semplice stampa, qui tipicamente ci sarà l'utilizzo dei dati per istanziare un oggetto del modello

```
java -cp /tmp/4HByiex4uA/HelloWorld
Giovanni Rossi|via Indipendenza 38|Bologna
Gian Paolo Prinzi|via Altabella 46|Bologna
Anna Maria Senzi|via dell'Arco 18/2|S.Lazzaro

=== Code Execution Successful ===
```



1° CASO – StringTokenizer

```
...  
// rdr è un BufferedReader  
while ((riga = rdr.readLine()) != null) {  
    StringTokenizer t = new StringTokenizer(riga, "$");  
    String nomeCognome = t.nextToken().trim();  
    String indirizzo = t.nextToken().trim();  
    String citta = t.nextToken().trim();  
    // costruzione modello a partire dai dati  
    list.add(new Persona(nomeCognome, indirizzo, citta));  
}  
...
```

Ad esempio, se il modello prevede la classe **Persona**, qui i dati saranno usati per istanziare una **Persona** e metterla in una collezione o array

1° CASO – Scanner

```
import java.util.Scanner;
import java.util.StringTokenizer;
import java.io.*;

class HelloWorld {
    public static void main(String[] args) throws IOException {
        var fakeFile = "Giovanni Rossi $   via Indipendenza 38$Bologna\n" +
            "Gian Paolo Prinzi  $ via Altabella 46$Bologna\n" +
            "Anna Maria Senzi$ via dell'Arco 18/2 $S.Lazzaro";

        var sc = new Scanner(fakeFile).useDelimiter("\\s*\\$\\s*|\\n");
        while(sc.hasNext()){
            var nomeCognome = sc.next();
            var indirizzo = sc.next();
            var citta = sc.next();
            System.out.println(nomeCognome + "|" + indirizzo + "|" + citta);
        }
    }
}
```

Aggiungendo `\\s*` evitiamo le `trim`

Cruciale l'escape
Altrimenti, BOOM!

Con Scanner, il delimitatore
deve prevedere, in OR,
anche il caso del fine riga (`\n`)

In luogo della semplice stampa, qui
tipicamente ci sarà l'utilizzo dei dati
per istanziare un oggetto del modello

```
java -cp /tmp/ZMp4KKj2aE/HelloWorld
Giovanni Rossi|via Indipendenza 38|Bologna
Gian Paolo Prinzi|via Altabella 46|Bologna
Anna Maria Senzi|via dell'Arco 18/2|S.Lazzaro

=== Code Execution Successful ===
```

1° CASO – split

```
import java.util.Scanner;
import java.util.StringTokenizer;
import java.io.*;

class HelloWorld {
    public static void main(String[] args) throws IOException {
        var fakeFile = "Giovanni Rossi $   via Indipendenza 38$Bologna\n" +
            "Gian Paolo Prinzi  $ via Altabella 46$Bologna\n" +
            "Anna Maria Senzi$ via dell'Arco 18/2 $S.Lazzaro";

        var reader = new BufferedReader(new StringReader(fakeFile));
        String riga;
        while((riga=reader.readLine())!=null){
            var items = riga.split("\\$");
            var nomeCognome = items[0].trim();
            var indirizzo = items[1].trim();
            var citta = items[2].trim();
            System.out.println(nomeCognome + "|" + indirizzo + "|" + citta);
        }
    }
}
```

Cruciale l'escape
Altrimenti... →

In luogo della semplice stampa, qui
tipicamente ci sarà l'utilizzo dei dati
per istanziare un oggetto del modello

```
java -cp /tmp/1UHrRtUFmo/HelloWorld
ERROR!
Exception in thread "main" java.lang.ArrayIndex
    Index 1 out of bounds for length 1
    at HelloWorld.main(HelloWorld.java:16)

=== Code Exited With Errors ===
```

```
java -cp /tmp/4HByiex4uA/HelloWorld
Giovanni Rossi|via Indipendenza 38|Bologna
Gian Paolo Prinzi|via Altabella 46|Bologna
Anna Maria Senzi|via dell'Arco 18/2|S.Lazzaro

=== Code Execution Successful ===
```

1° CASO – split

```
...  
// rdr è un BufferedReader  
while ((riga = rdr.readLine()) != null) {  
    var items = riga.split("\\s*\\$\\s*");  
    var nomeCognome = items[0];  
    var indirizzo = items[1];  
    var citta = items[2];  
    list.add(  
        new Persona(nomeCognome, indirizzo, città));  
}  
...
```

Aggiungendo `\\s*` possiamo evitare le successive `trim`

Cruciale la sequenza di escape `\\$` perché `$` è un simbolo speciale: senza `\\`, significa fine riga e in tal caso il parsing fallisce!

Ad esempio, se il modello prevede la classe `Persona`, qui i dati saranno usati per istanziare una `Persona` e metterla in una collezione o array

2° CASO

Esempio 2: *separatori diversi, ma potenzialmente assimilabili*

Giovanni Rossi, via Indipendenza 38 - Bologna

Gian Paolo Prinzi, viale dei Mille 42 - Reggio Emilia

Anna Maria Senzi, via dell'Arco 18/2- S.Lazzaro

- Punto chiave: qui la «virgola» NON compare come testo all'interno del secondo token
- Se non interessa verificare con precisione che ci sia proprio una virgola fra i primi due token e proprio un trattino fra i secondi due, ciò permette di *specificare direttamente due delimitatori insieme* (, | -) +, come se fossero uno solo 😊
 - di fatto si torna al caso precedente, con minime modifiche



2° CASO con StringTokenizer

```
import java.util.Scanner;
import java.util.StringTokenizer;
import java.io.*;

class HelloWorld {
    public static void main(String[] args) throws IOException {
        var fakeFile = "Giovanni Rossi ,   via Indipendenza 38-Bologna\n" +
                       "Gian Paolo Prinzi  , via Altabella 46 - Bologna\n" +
                       "Anna Maria Senzi, via dell'Arco 18/2 -- S.Lazzaro";

        var reader = new BufferedReader(new StringReader(fakeFile));
        String riga;
        while((riga=reader.readLine())!=null){
            var stk = new StringTokenizer(riga, ",-");
            var nomeCognome = stk.nextToken().trim();
            var indirizzo = stk.nextToken().trim();
            var citta = stk.nextToken().trim();
            System.out.println(nomeCognome + "|" + indirizzo
                               + "|" + citta);
        }
    }
}
```

**Due delimitatori specificati
insieme in OR fra loro**

```
java -cp /tmp/vLcF2sVKyG/HelloWorld
Giovanni Rossi|via Indipendenza 38|Bologna
Gian Paolo Prinzi|via Altabella 46|Bologna
Anna Maria Senzi|via dell'Arco 18/2|S.Lazzaro

=== Code Execution Successful ===
```



2° CASO con Scanner

```
import java.util.Scanner;
import java.util.StringTokenizer;
import java.io.*;

class HelloWorld {
    public static void main(String[] args) throws IOException {
        var fakeFile = "Giovanni Rossi ,   via Indipendenza 38-Bologna\n" +
                       "Gian Paolo Prinzi   , via Altabella 46 - Bologna\n" +
                       "Anna Maria Senzi, via dell'Arco 18/2 -- S.Lazzaro";

        var sc = new Scanner(fakeFile).useDelimiter("\\s*(,|-)+\\s*\\n");
        while(sc.hasNext()){
            var nomeCognome = sc.next();
            var indirizzo = sc.next();
            var citta = sc.next();
            System.out.println(nomeCognome + "|" + indirizzo + "|" + citta);
        }
    }
}
```

Due delimitatori specificati insieme
in OR fra loro (e col newline)

```
java -cp /tmp/vLcF2sVKyG/HelloWorld
Giovanni Rossi|via Indipendenza 38|Bologna
Gian Paolo Prinzi|via Altabella 46|Bologna
Anna Maria Senzi|via dell'Arco 18/2|S.Lazzaro

=== Code Execution Successful ===
```

2° CASO con split

```
import java.util.Scanner;
import java.util.StringTokenizer;
import java.io.*;

class HelloWorld {
    public static void main(String[] args) throws IOException {
        var fakeFile = "Giovanni Rossi ,   via Indipendenza 38-Bologna\n" +
                       "Gian Paolo Prinzi   , via Altabella 46 - Bologna\n" +
                       "Anna Maria Senzi, via dell'Arco 18/2 -- S.Lazzaro";

        var reader = new BufferedReader(new StringReader(fakeFile));
        String riga;
        while((riga=reader.readLine())!=null){
            var items = riga.split("\\s*(,|-)+\\s*");
            var nomeCognome = items[0];
            var indirizzo = items[1];
            var citta = items[2];
            System.out.println(nomeCognome + "|" + indirizzo
                               + "|" + citta);
        }
    }
}
```

Due delimitatori specificati
insieme in OR fra loro

```
java -cp /tmp/vLcF2sVKyG/HelloWorld
Giovanni Rossi|via Indipendenza 38|Bologna
Gian Paolo Prinzi|via Altabella 46|Bologna
Anna Maria Senzi|via dell'Arco 18/2|S.Lazzaro

=== Code Execution Successful ===
```

3° CASO

Esempio 3: *separatori diversi, da trattare distintamente*

Giovanni Rossi, via Indipendenza, 38 - Bologna
Gian Paolo Prinzi, viale dei Mille, 42 - Reggio Emilia
Anna Maria Senzi, via dell'Arco, 18/2 - S.Lazzaro

- Qui *il primo delimitatore (la «virgola») compare anche come testo all'interno del secondo token*
- Ciò costringe a fare il parsing in due tempi:
 - prima la parte separata da virgola
 - poi, *cambiando delimitatore*, quella separata da lineetta
- Più facile con **StringTokenizer** e **Scanner**, dove si possono cambiare i delimitatori al volo; più contorto con **split**

3° CASO

- Punto di attenzione: *cambiando delimitatore, il precedente delimitatore viene declassato a carattere «standard»* e come tale incluso nel successivo token ☹, che risulta così «sporcato» e «inquinato» da un carattere spurio
- Possibili soluzioni:
 - con **StringTokenizer**: farsi restituire *anche i delimitatori come token* (c'è un costruttore apposito, a tre argomenti) e poi buttarli
 - ricorrere a elaborazioni manuali di stringhe ☹
 - con **Scanner** : ricorrere a pattern «furbi» nel delimitatore 😊😊

3° CASO

con StringTokenizer

```
class HelloWorld {  
    public static void main(String[] args) throws IOException {  
        var fakeFile = "Giovanni Rossi ,   via Indipendenza, 38-Bologna\n" +  
                        "Gian Paolo Prinzi  , via Altabella, 46 - Bologna\n" +  
                        "Anna Maria Senzi, via dell'Arco, 18/2 -- S.Lazzaro";  
  
        var reader = new BufferedReader(new StringReader(fakeFile));  
        String riga;  
        while((riga=reader.readLine())!=null){  
            var stk = new StringTokenizer(riga, ",");  
            var nomeCognome = stk.nextToken().trim();  
            var indirizzo = stk.nextToken("-").trim();  
            var citta = stk.nextToken("\n\r").trim();  
            System.out.println(nomeCognome + "|" + indirizzo + "|" + citta);  
        }  
    }  
}
```

la virgola torna ad essere
un carattere normale

Il trattino torna ad essere
un carattere normale

```
java -cp /tmp/mq7mV5aEor/HelloWorld  
Giovanni Rossi ,   via Indipendenza, 38 -Bologna  
Gian Paolo Prinzi|, via Altabella, 46| - Bologna  
Anna Maria Senzi|, via dell'Arco, 18/2 -- S.Lazzaro
```

La virgola viene quindi
inclusa nel 2° token

Il trattino viene quindi
incluso nel 3° token



3° CASO

con StringTokenizer

```
class HelloWorld {  
    public static void main(String[] args) throws IOException {  
        var fakeFile = "Giovanni Rossi ,   via Indipendenza, 38-Bologna\n" +  
            "Gian Paolo Prinzi   , via Altabella, 46 - Bologna\n" +  
            "Anna Maria Senzi,via dell'Arco, 18/2 - S.Lazzaro";  
  
        var reader = new BufferedReader(new StringReader(fakeFile));  
        String riga;  
        while((riga=reader.readLine())!=null){  
            var stk = new StringTokenizer(riga, ",", true);  
            var nomeCognome = stk.nextToken().trim();  
            stk.nextToken().trim(); // butta via la virgola  
            var indirizzo = stk.nextToken("-").trim();  
            stk.nextToken().trim(); // butta via il trattino  
            var citta = stk.nextToken("\n\r").trim();  
            System.out.println(nomeCognome + "|" + indirizzo + "|" + citta);  
        }  
    }  
}
```

Flag **returnsDelim = true**

Restituisce come token *anche ogni singolo carattere delimitatore*

MA questa soluzione funziona SOLO se c'è UN SINGOLO carattere delimitatore

```
java -cp /tmp/WQrN7qNxy5/HelloWorld  
Giovanni Rossi|via Indipendenza, 38|Bologna  
Gian Paolo Prinzi|via Altabella, 46|Bologna  
Anna Maria Senzi|via dell'Arco, 18/2|S.Lazzaro  
  
=== Code Execution Successful ===
```



3° CASO

con StringTokenizer

```
var fakeFile = "Giovanni Rossi ,   via Indipendenza, 38-Bologna\n" +  
               "Gian Paolo Prinzi, , via Altabella, 46 - Bologna\n" +  
               "Anna Maria Senzi,via dell'Arco, 18/2 -- S.Lazzaro";
```

Se ci sono più delimitatori in sequenza, ognuno di essi dà luogo a un *token a sé stante* → consumarne uno solo non basta ☹, perché l'altro sopravvive → servirebbe un ciclo...

```
java -cp /tmp/ROVTweGpt9/HelloWorld  
Giovanni Rossi|via Indipendenza, 38|Bologna  
Gian Paolo Prinzi, |via Altabella, 46|Bologna  
Anna Maria Senzi|via dell'Arco, 18/2|- S.Lazzaro  
  
=== Code Execution Successful ===
```

Morale:

- **StringTokenizer** è troppo basico per elaborazioni complesse
- con **Scanner** le cose miglioreranno perché l'espressività del pattern di separazione è nettamente superiore:
in particolare, `\\^` consente di esprimere anche la *negazione* ☺



3° CASO con Scanner

- L'espressività di **Scanner** permette di:
 - definire in modo preciso i delimitatori da usare volta per volta, con `useDelimiter(delimiterPattern)`
 - specificare il token da cercare, con `next(tokenPattern)`
 - eventualmente, saltare alcune parti, con `skip(skipPattern)`
- Nel nostro caso, quindi, dovremo:
 - definire inizialmente il delimitatore «virgola» (con spazi attorno)
 - definire il token da cercare come composto solo di caratteri «diversi da virgola»
 - saltare la virgola stessa (con eventuali spazi attorno)
 - ripetere tutto per il trattino, per il 2° token
 - ripetere tutto con il newline, per il 3° token

3° CASO con Scanner

...

```
var sc = new Scanner(fakeFile);  
while (sc.hasNext()) {  
    sc.useDelimiter("\\s*,+\\s*|\\n");  
    var nomeCognome = sc.next("[^,]*");  
    sc.skip("(\\s*,\\s*)*");  
    sc.useDelimiter("\\s*(-)+\\s*|\\n");  
    var indirizzo = sc.next("[^-]*");  
    sc.skip("(\\s*-\\s*)*");  
    sc.useDelimiter("\\n");  
    var citta = sc.next();  
    ...  
}
```

Delimitatore iniziale: una o più virgole (o newline)

1° token: qualunque sequenza di caratteri ≠ virgola

Salta le eventuali virgole (con spazi attorno), 0+ volte

Delimitatore modificato in 1+ trattini (o newline)

2° token: qualunque sequenza di caratteri ≠ trattino

Salta gli eventuali trattini (con spazi attorno), 0+ volte

3° token: tutti i restanti caratteri fino a fine riga



3° CASO con Scanner

```
class HelloWorld {  
    public static void main(String[] args) throws IOException {  
        var fakeFile = "Giovanni Rossi ,   via Indipendenza, 38-Bologna\n" +  
                        "Gian Paolo Prinzi,   , via Altabella, 46 - Bologna\n" +  
                        "Anna Maria Senzi,via dell'Arco, 18/2 -- S.Lazzaro";  
  
        var sc = new Scanner(fakeFile);  
        while(sc.hasNext()){  
            sc.useDelimiter("\\s*,+\\s*|\\n");  
            var nomeCognome = sc.next("[^,]*");  
            sc.skip("(\\s*,\\s*)*");  
            sc.useDelimiter("\\s*(-)+\\s*|\\n");  
            var indirizzo = sc.next("[^-]*");  
            sc.skip("(\\s*-\\s*)*");  
            sc.useDelimiter("\\n");  
            var citta = sc.next();  
            System.out.println(nomeCognome + "|" + indirizzo + "|" + citta);  
        }  
    }  
}
```

```
java -cp /tmp/z04cdMrT8i/HelloWorld  
Giovanni Rossi|via Indipendenza, 38|Bologna  
Gian Paolo Prinzi|via Altabella, 46|Bologna  
Anna Maria Senzi|via dell'Arco, 18/2|S.Lazzaro  
  
=== Code Execution Successful ===
```



3° CASO con `split`

- L'espressività di `split` è inferiore a quella di **Scanner**:
 - permette solo di **definire in modo preciso i delimitatori, che però saranno poi usati a tappeto ovunque**
 - NON consente di specificare il token da cercare, né di saltare parti
- Nel nostro caso, poiché la «virgola» è anche dentro a un token mentre il trattino no, conviene partire da quest'ultimo:
 - FASE A: splittiamo la riga con il **delimitatore «trattino»**, ottenendo:
 - come 1^a parte, «nome cognome e indirizzo»
 - come 2^a parte, la città
 - FASE B: splittiamo ora quella prima parte con il **delimitatore «virgola»** (con spazi attorno), ottenendo *tre pezzi*:
 - come 1^a parte, «nome cognome»
 - come 2^a e 3^a parte, separatamente, indirizzo e n° civico

3° CASO con split

```
...  
// rdr è un BufferedReader  
while ((riga = rdr.readLine()) != null) {  
    var items = riga.split("\\s*~+\\s*|\\n");  
    var firstPart = items[0];  
    var citta = items[1];  
    items = firstPart.split("(\\s*,+\\s*)+|\\n");  
    var nomeCognome = items[0];  
    var indirizzo =  
        items[1] + ", " + items[2];  
    ...  
}
```

Split iniziale: uno o più **trattini** (o newline)

Nome cognome & indirizzo

città

Nome cognome

Split successivo: una o più **virgole** (o newline)

Indirizzo

N° civico

Solo se necessario: ri-montaggio manuale dei pezzi separati (ma meglio sarebbe *separare indirizzo e n° civico anche nel model..!!*)

3° CASO con split

```
class HelloWorld {  
    public static void main(String[] args) throws IOException {  
        var fakeFile = "Giovanni Rossi ,   via Indipendenza, 38-Bologna\n" +  
            "Gian Paolo Prinzi,   , via Altabella, 46 - Bologna\n" +  
            "Anna Maria Senzi,via dell'Arco, 18/2 -- S.Lazzaro";  
  
        var reader = new BufferedReader(new StringReader(fakeFile));  
        String riga;  
        while((riga=reader.readLine())!=null){  
            var items = riga.split("(\\s*~+\\s*)+|\\n");  
            var firstPart = items[0];  
            var citta = items[1];  
            items = firstPart.split("(\\s*,+\\s*)+|\\n");  
            var nomeCognome = items[0];  
            var indirizzo = items[1] + ", " + items[2];  
            System.out.println(nomeCognome + "|" + indirizzo + "|" + citta);  
        }  
    }  
}
```

Solo se necessario: ri-montaggio manuale dei pezzi separati (ma meglio sarebbe *separare indirizzo e n° civico anche nel model..!!*)

```
java -cp /tmp/Q88z07G4WE/HelloWorld  
Giovanni Rossi|via Indipendenza, 38|Bologna  
Gian Paolo Prinzi|via Altabella, 46|Bologna  
Anna Maria Senzi|via dell'Arco, 18/2|S.Lazzaro  
  
=== Code Execution Successful ===
```

4° CASO

Esempio 4: *separatori impliciti*

alfabetico *alfabetico* *numerico*

Madre Arianna3471234567

Padre Giovanni3349876543

- Nessun separatore compare come testo all'interno di qualche token → ottimo
- *Non è detto che ci siano spazi fra il secondo e il terzo token*
→ separatori anche impliciti → lavoro in più fasi:
 - prima, fino al primo «spazio»
 - poi, fino al primo «carattere numerico»
 - infine, fino a fine riga



4° CASO con StringTokenizer

```
class HelloWorld {  
    public static void main(String[] args) throws IOException {  
  
        String fakeFile = "Madre Arianna3471234567\n" +  
                           "Padre Giovanni3349876543";  
  
        var rdr = new BufferedReader(new StringReader(fakeFile));  
  
        String riga;  
        while ((riga = rdr.readLine()) != null) {  
            StringTokenizer t = new StringTokenizer(riga);  
            var ruolo = t.nextToken().trim();  
            var nome = t.nextToken("0123456789").trim();  
            var telefono = t.nextToken(" \t\n\r").trim();  
            System.out.println(ruolo + "|" + nome + "|" + telefono);  
        }  
    }  
}
```

separatori iniziali = spazi

separatori = cifre numeriche

separatori = fine riga

```
java -cp /tmp/MvqI3oS3Ux/HelloWorld  
Madre|Arianna|3471234567  
Padre|Giovanni|3349876543  
  
=== Code Execution Successful ===
```

4° CASO con Scanner

```
class HelloWorld {  
    public static void main(String[] args) throws IOException {  
  
        String fakeFile = "Madre Arianna3471234567\n" +  
                           "Padre Giovanni3349876543";  
  
        var sc = new Scanner(fakeFile);  
  
        while (sc.hasNext()) {  
            var ruolo = sc.next();  
            sc.skip("\\s+");  
            sc.useDelimiter("\\s*\\d+\\s*");  
            var nome = sc.next();  
            sc.reset();  
            var telefono = sc.next();  
            System.out.println(ruolo + "|" + nome + "|" + telefono);  
        }  
    }  
}
```

separatori iniziali = spazi

Consuma gli spazi

separatori = cifre numeriche

separatori = default

```
java -cp /tmp/7c5Bsv1WlX/HelloWorld  
Madre|Arianna|3471234567  
Padre|Giovanni|3349876543
```

=== Code Execution Successful ===

4° CASO con split

```
class HelloWorld {  
    public static void main(String[] args) throws IOException {  
  
        String fakeFile = "Madre Arianna3471234567\n" +  
                           "Padre Giovanni3349876543";  
  
        var rdr = new BufferedReader(new StringReader(fakeFile));  
  
        String riga;  
        while ((riga = rdr.readLine()) != null) {  
            var items = riga.split("\\s+");  
            var ruolo = items[0].trim();  
            var others = items[1].trim();  
            items = others.split("\\s*\\d+\\s*");  
            var nome = items[0].trim();  
            items = others.split("\\s*\\D+\\s*");  
            var telefono = items[1].trim();  
            System.out.println(ruolo + "|" + nome + "|" + telefono);  
        }  
    }  
}
```

```
java -cp /tmp/7c5Bsv1WlX/HelloWorld  
Madre|Arianna|3471234567  
Padre|Giovanni|3349876543  
=== Code Execution Successful ===
```

separatori iniziali = spazi

1° token = ruolo, 2° token = resto della riga

separatori = cifre numeriche

1° token = nome, 2° token = resto della riga

separatori = cifre NON numeriche

1° token = vuoto, 2° token = telefono

5° CASO

Esempio 5: *larghezza fissa (non servono separatori)*

6 char

8 char

10 char

Madre Arianna 3471234567

FiglioGiovanni3349876543

- In questi casi non servono tokenizzatori: conviene *separare le parti semplicemente con substring*
 - RICORDA: gli indici di substring sono chiusi a sx e aperti a dx
 - ciò permette di «concatenare visivamente» le estrazioni
- ESEMPIO:

```
var ruolo = riga.substring(0,6).trim();  
var nome  = riga.substring(6,14).trim();  
var tel   = riga.substring(14).trim();
```



UN PRIMO BILANCIO

- **StringTokenizer** è più semplice per estrarre frammenti di stringhe con delimitatori variegati
 - MA non avendo shortcut per classi di caratteri, diventa molto scomodo se ci sono molti caratteri di separazione da specificare
- **Scanner** è vincente per situazioni più complesse e variegate, o per estrarre valori primitivi con delimitatori mix
 - permette di specificare delimitatori e pattern con *espressioni regolari*, potenti e compatte (ma più complesse da usare)
 - attenzione ai caratteri speciali e ai necessari escape!
 - possibile evitare `trim` grazie a pattern più complessi con spazi
- **split** è molto comodo nei casi di delimitatore fisso
 - attenzione ai caratteri speciali e ai necessari escape!

Tokenizzazione di valori formattati: valori numerici, date e orari



TOKENIZZAZIONE E LETTURA DI VALORI NUMERICI

- Dopo aver acquisito il singolo token (una stringa), se si tratta di un valore numerico (`int`, `double`, etc.) o di prezzi/valute o percentuali, è quasi sempre necessario **convertirlo in un valore del giusto tipo**
 - MOTIVO: il valore servirà per costruire oggetti o fare calcoli
- Si usano i metodi **parse** dei *formattatori numerici*
 - CRUCIALE configurare il formattatore con la giusta *cultura locale*!
- Punti di attenzione:
 - per i valori **floating point**: quale separatore per la parte decimale? occorre verificare il separatore delle migliaia...?
 - per i **prezzi**: il simbolo di valuta è davanti o dietro? serve un formattatore personalizzato o vanno bene quelli standard?



PARSING DI STRINGHE NUMERICHE

- I formattatori creati da **NumberFormat** espongono un metodo **parse** per convertire una **stringa numerica** in un **valore numerico** di classe **Number**
 - la stringa dev'essere *correttamente formattata* come **numero**, **percentuale** o **valuta** secondo le regole della *cultura locale* scelta
 - il numero convertito è poi estraibile come **double**, **float** o **int**

```
jshell> import java.text.*
jshell> var nf = NumberFormat.getNumberInstance(Locale.ITALY)
```

```
jshell> Number number = nf.parse("1.234,56")
number ==> 1234.56
```

Formattatore per valore numerici standard

```
jshell> number.doubleValue()
$4 ==> 1234.56
```

Il metodo parse restituisce un oggetto di classe **Number** (la classe astratta capostipite dei wrapper numerici). Appositi metodi consentono di estrarre il valore come **double**, **float**, **int**, etc.

```
jshell> number.intValue()
$5 ==> 1234
```

PARSING DI STRINGHE NUMERICHE

- Ulteriore esempio con valute e prezzi

```
jshell> import java.text.*
jshell> var cf = NumberFormat.getCurrencyInstance(Locale.ITALY)

jshell> Number number = nf.parse("1.234,56 €")
number ==> 1234.56
```

Formattatore per prezzi

Il formattatore estrae il valore numerico puro, verificando la presenza del (e poi eliminando) simbolo di valuta

In questo caso la stringa contiene un prezzo, comprensivo del simbolo di valuta previsto per quel Locale (qui Euro, dopo il valore)

- Con formattatore personalizzato:

```
jshell> import java.text.*
jshell> var f = new DecimalFormat("¤ ###0.##")

jshell> f.parse("€ 123,45")
$23 ==> 123.45
```

Formattatore personalizzato

Il simbolo di valuta davanti è ora tranquillamente accettato

PARSING DI STRINGHE NUMERICHE

- Con formattatore personalizzato per valori anche negativi:

```
jshell> import java.text.*
jshell> var f = new DecimalFormat("¤ ###0.##;¤ -###0.##")

jshell> f.parse("€ -123,45")
$23 ==> -123.45
```

Formattatore personalizzato

Il simbolo di valuta davanti è ora
accettato anche per prezzi negativi

NB: la posizione del segno «-» è a
scelta, dipende dal pattern indicato

- Attenzione, però:
il formattatore personalizzato usa il *Locale* di default !
 - ciò può portare a risultati inaspettati
 - per impostare il **Locale**, appoggiarsi a **DecimalFormatSymbols**
→ **`new DecimalFormat("¤ ###0.##",
new DecimalFormatSymbols(Locale.ITALY))`**



PARSING DI STRINGHE NUMERICHE

- Analoghi problemi sorgono con le percentuali

```
jshell> import java.text.*  
jshell> var pf = NumberFormat.getPercentInstance(Locale.ITALY)  
  
jshell> Number number = pf.parse("36,75%")  
number ==> 0.3675
```

Formattatore per percentuali

Qui la stringa contiene una percentuale

Il formattatore estrae il valore numerico puro, dividendo il valore per 100



PARSING DI STRINGHE NUMERICHE: problemi

- Attenzione alle situazioni problematiche
 - i formattatori numerici **ignorano il simbolo delle migliaia**, ovunque sia
 - questo può portare a **risultati assurdi**, come percentuali «enormi», valori «molto più grandi» del previsto, etc., che però **rischiano di passare inosservati** perché «apparentemente ok»

```
jshell> import java.text.*  
jshell> var nf = NumberFormat.getNumberInstance(Locale.ITALY)
```

```
jshell> Number number = nf.parse("3.675")  
number ==> 3675
```

Separatore delle migliaia
dove ce lo aspetteremmo...

```
jshell> Number number = nf.parse("36.75")  
number ==> 3675
```

... e dove no: **ma viene
ignorato!** Quindi, occhio!

```
jshell> Number number = nf.parse("367.5")  
number ==> 3675
```



PARSING DI STRINGHE NUMERICHE: problemi

- Questo problema *non* si risolve neppure usando il metodo `parse` a due argomenti (con `ParsePosition`) perché *il simbolo delle migliaia mal posizionato non costituisce comunque un errore* (è semplicemente ignorato)
- L'unica via è, quindi (purtroppo), il *controllo manuale* ☹
 - si recupera il simbolo delle migliaia dal formattatore
 - poi, lo si *cerca ed elimina manualmente* con `String.replace`

```
jshell> var pos = new ParsePosition(0)
pos ==> java.text.ParsePosition[index=0,errorIndex=-1]
```

```
jshell> Number number = nf.parse("367.5", pos)
number ==> 3675
```

```
jshell> pos
pos ==> java.text.ParsePosition[index=5,errorIndex=-1]
```

La parsing position è arrivata comunque in fondo normalmente



PARSING DI STRINGHE NUMERICHE: problemi

- Si recupera il simbolo delle migliaia (grouping symbol)

```
jshell> var sym = ((DecimalFormat)nf).getDecimalFormatSymbols()  
                        .getGroupingSeparator()
```

sym ==> '.'

Un carattere

Carattere convertito
in stringa

...e lo si toglie a tappeto dalla stringa:

```
jshell> var number = nf.parse("367.5".replace(String.valueOf(sym), ""))  
number ==> 3675
```

- Altro caso critico: le percentuali

```
jshell> import java.text.*  
jshell> var pf = NumberFormat.getPercentInstance(Locale.ITALY)
```

```
jshell> Number number = pf.parse("36.75%")  
number ==> 36.75
```

Formattatore per percentuali

Il formattatore ignora il punto decimale, considera il valore come 3675% e, dividendolo (correttamente) per 100, dà come risultato un assai improbabile 36.75... ARGHH!!

OCCHIO AL PUNTO DECIMALE invece della prevista virgola!



PARSING DI STRINGHE NUMERICHE: problemi

- Se invece il formato è errato, scatta un'eccezione
 - spazi imprevisti, caratteri inammissibili, etc.
- Per questo, **parse** richiede il **try/catch** obbligatorio
 - può sembrare una scocciatura, ma in realtà è meglio che ritrovarsi con valori assurdi (come nei casi precedenti) «senza accorgersene»
- Caso critico: le percentuali

```
jshell> import java.text.*
jshell> var pf = NumberFormat.getPercentInstance()
jshell> Number number = pf.parse("36,75 %")
| Exception java.text.ParseException: Unparseable number: "36,75 %"
|     at NumberFormat.parse (NumberFormat.java:434)
|     at (#11:1)
```

SPAZIO EXTRA NON PREVISTO
prima del simbolo di percentuale



TOKENIZZAZIONE E LETTURA DI DATE E ORARI

- Se la stringa rappresenta una data e/o orario, è necessario **convertirla in un oggetto del giusto tipo**
- Si usano gli appositi **formattatori per date e orari**
 - anche qui, occhio alla cultura **Locale** e relative convenzioni
 - occhio allo *stile* (short, medium, long, full) e ai relativi mix
 - ricordare che date nella forma GG/MM/AAAA non sono supportate dagli stili standard, quindi occorre farsi un *formattatore personalizzato* con `ofPattern("dd/MM/yyyy")`
- Ci sono **due possibili approcci**:
 - **o si chiede al formattatore** di fare il parsing, convertendo il risultato nel giusto **LocalDate/LocalTime** col metodo statico **from**
 - **oppure lo si chiede a queste ultime**, tramite il metodo statico **parse** (passando loro il formattatore da usare)



TOKENIZZAZIONE E LETTURA DI DATE E ORARI

- APPROCCIO #1: partire dal formattatore
 - prima si chiede al formattatore di fare il parsing
 - poi si converte il risultato (un generico **TemporalAccessor**) nel corretto tipo di oggetto (**LocalDate/-Time**)

```
var t = formatter.parse(stringa);  
var d = LocalDate.from(t);
```
- APPROCCIO #2: partire dalla data/orario
 - premessa: il formattatore dev'essere comunque predisposto
 - si chiede alla classe data/orario di fare il parsing, passandole come argomento non solo la stringa, ma anche il formatter da usare

```
var d = LocalDate.parse(stringa, formatter);
```



PARSING DI DATE E ORARI

Approccio #1

- I formattatori creati da `DateTimeFormatter.ofXXX` espongono vari metodi **parse**
 - il metodo **parse** basico restituisce un **TemporalAccessor**, che è una sorta di «contenitore generico» per oggetti «temporali»
 - il risultato va perciò **convertito nel tipo di oggetto atteso** con i **metodi from** presenti in **LocalDate**, **LocalTime**, **LocalDateTime**

```
jshell> import java.time.*
jshell> import java.time.format.*
jshell> import java.time.temporal.*

jshell> var fmt = DateTimeFormatter.ofLocalizedDate(FormatStyle.SHORT)
                                .withLocale(Locale.ITALY)

jshell> var t = fmt.parse("12/02/23")

jshell> LocalDate d = LocalDate.from(t)
d ==> 2023-02-12
```

Restituisce un
TemporalAccessor...

...da usare poi per costruire la
data col metodo-factory **from**



PARSING DI DATE E ORARI

Approccio #1

- Un altro esempio

```
jshell> var fmt = DateTimeFormatter.ofLocalizedDateTime(  
    FormatStyle.SHORT, FormatStyle.SHORT)  
    .withLocale(Locale.ITALY)  
  
jshell> var tp = fmt.parse("12/02/23, 12:40")  
  
jshell> LocalDate d = LocalDate.from(tp)  
d ==> 2023-02-12  
  
jshell> LocalTime t = LocalTime.from(tp)  
t ==> 12:40  
  
jshell> LocalDateTime dt = LocalDateTime.from(tp)  
dt ==> 2023-02-12T12:40
```

Restituisce il solito
TemporalAccessor

Essendo una data/ora, si
possono estrarre a piacimento
o solo la data, o solo l'ora, o
entrambe

– nei casi banali, ovviamente, si può anche fare tutto insieme:

```
jshell> var dt = LocalDateTime.from(fmt.parse("12/02/23, 12:40"))
```



PARSING DI DATE E ORARI

Approccio #2

- In alternativa, si possono sfruttare i metodi **parse** esposti da **LocalDate**, **LocalTime**, etc.
 - in questo caso, **il formattatore che governa la conversione viene passato come (secondo) argomento** al metodo **parse** stesso

```
jshell> var dt = LocalDateTime.parse("12/02/23, 12:30", fmt)
dt ==> 2023-02-12T12:30

jshell> var d = LocalDate.parse("12/02/23, 12:30", fmt)
d ==> 2023-02-12

jshell> var t = LocalTime.parse("12/02/23, 12:30", fmt)
t ==> 12:30
```

Si invoca il metodo **parse** della classe che interessa

Il formattatore è il secondo argomento del metodo **parse**

- ovviamente, **il formattatore dev'essere coerente con la stringa passata**
→ nell'esempio sopra, dev'essere adatto per **LocalDateTime**
- però, il metodo **parse** invocato può anche essere quello di **LocalDate** o **LocalTime**, se interessa solo la data o solo l'ora

Qualche caso di compito d'esame



COMPITI D'ESAME

Statistiche treni, Media degli esami

Ogni riga descrive la fermata del treno in una stazione, specificando nell'ordine il *nome della stazione*, l'*orario di arrivo previsto*, l'*orario di arrivo effettivo*, l'*orario di partenza previsto* e l'*orario di partenza effettivo*. Questi valori sono separati uno dall'altro da uno o più caratteri "punto e virgola" (';'): non è dato sapere se vi siano spazi intorno, prima, davanti ai vari campi. Tutti gli orari sono in formati italiano SHORT.

Nel caso della stazione iniziale, naturalmente, gli orari di arrivo sono indefiniti; lo stesso accade nell'ultima stazione per gli orari di partenza. Al loro posto, per convenzione, è indicata una sequenza di almeno due trattini.

ESEMPIO

```
MILANO CENTRALE;-- ;-- ;13:20;13:54
MILANO LAMBRATE;13:26;13:59;13:27;14:01
MILANO ROGOREDO;13:31;14:05;13:32;14:07
...
ANZOLA ;15:56;16:13;15:57;16:14
BOLOGNA C.LE;16:10;16:24;-- ;--
```

ESEMPIO:

27991	ANALISI MATEMATICA T-1	9,0	12/1/2020	RT
27991	ANALISI MATEMATICA T-1	9,0	10/2/2020	22
28004	FONDAMENTI DI INFORMATICA T-1	12,0	13/2/2020	24
29228	GEOMETRIA E ALGEBRA T	6,0	18/1/2020	26
26337	LINGUA INGLESE B-2	6,0	18/6/2020	ID
27993	ANALISI MATEMATICA T-2	6,0	10/6/2020	RE
27993	ANALISI MATEMATICA T-2	6,0	02/7/2020	RT
27993	ANALISI MATEMATICA T-2	6,0	22/7/2020	23
28006	FONDAMENTI DI INFORMATICA T-2	12,0	21/7/2020	27
28011	RETI LOGICHE T	6,0	22/2/2022	22
28012	CALCOLATORI ELETTRONICI T	6,0	22/1/2021	RT
28012	CALCOLATORI ELETTRONICI T	6,0	22/2/2021	20



COMPITI D'ESAME

Spese sanitarie, Sudoku

Il file di testo [spesesanitarie.txt](#) contiene i dati di una serie di documenti di spesa, uno dietro l'altro. Ogni documento di spesa è organizzato su più righe: in tutte, gli elementi sono separati dal carattere "punto e virgola" (;).

La prima riga specifica quattro elementi:

- la data di emissione del documento, nel formato GG/MM/AAAA;
- l'emittente della prestazione (una stringa);
- il numero (intero) di voci di cui il documento stesso è composto, non inferiore a 1;
- il totale in Euro del documento stesso, formattato col simbolo di valuta *davanti* al valore numerico, separato da uno spazio; a sua volta, il valore numerico è espresso secondo le convenzioni italiane.

Le righe successive (in numero uguale al numero di voci precedentemente specificato) riportano ciascuna:

- la tipologia della spesa (una sigla di due lettere identica alle costanti dell'enumerativo **Tipologia**);
- il relativo importo in Euro, formattato come sopra.

ESEMPIO

```
25/01/2022;Dentista;1;€ 300,00
LP;€ 300,00
08/02/2022;Farmacia;4;€ 43,98
FC;€ 10,77
FC;€ 6,03
FC;€ 19,62
FC;€ 7,56
...
```

Lo schema di gioco è descritto nel file di testo [sudokuConfig.txt](#) formattato come segue:

- ogni riga nel file di testo descrive una riga della scacchiera del sudoku, che a sua volta conterrà i dati delle rispettive colonne (nel caso della scacchiera classica 9x9, ci saranno quindi 9 righe ognuna delle quali descriverà 9 colonne): è esplicitamente richiesto di non cablare soluzione sulla dimensione 9 del sudoku tradizionale
- in ogni riga i valori numerici sono separati fra loro da tabulazioni, mentre le celle vuote sono rappresentate dal carattere "#"

Esempio:

```
7 # # 5 8 # # # 3
# # 4 # # # # 2 #
...
```



COMPITI D'ESAME

Previsioni del tempo, Elezioni

Il file di testo [previsioni.txt](#) contiene una serie di previsioni, una per riga, riferite a più località ed eventualmente per diverse date: in ogni riga, gli elementi sono separati da virgole.

Ogni riga specifica cinque elementi:

- la località (una stringa)
- la data a cui la previsione si riferisce, nel classico formato GG/MM/AA
- l'orario a cui la previsione si riferisce, nel classico formato HH:MM
- la temperatura prevista, costituita da un numero intero (eventualmente anche negativo) subito seguito, senza spazi intermedi, dal carattere '°'
- la probabilità di pioggia prevista, costituita da un numero intero (ovviamente compreso fra 0 e 100) subito seguito, senza spazi intermedi, dal carattere '%'

ESEMPIO

```
Bologna, 12/07/22, 02:00, 18°, 91%
Bologna, 12/07/22, 05:00, 16°, 92%
Bologna, 12/07/22, 08:00, 20°, 87%
Bologna, 12/07/22, 11:00, 24°, 64%
...
Ferrara, 18/07/22, 05:00, 18°, 91%
Ferrara, 18/07/22, 15:00, 16°, 92%
Ferrara, 18/07/22, 22:00, 20°, 87%
Reggio Emilia, 11/07/22, 02:00, -1°, 21%
Reggio Emilia, 11/07/22, 05:00, 16°, 92%
Reggio Emilia, 11/07/22, 08:00, 20°, 87%
...
```

Persistenza (dentinia.governor.persistence)

(punti 11)

Il file di testo [Voti.txt](#) contiene i risultati delle elezioni: la prima riga indica il numero di seggi totali da assegnare, nella forma "SEGGI" seguita da un intero, mentre la successiva riporta i voti ottenuti da ciascun partito, separati da virgole: ogni coppia partito/voti prevede prima il nome del partito (che può contenere spazi), poi i voti, formattati secondo l'uso italiano con il "." come separatore delle migliaia.

SEGGI 20

Lista A 9.100.000, Lista E 1.320.000, Lista C 4.880.000, Lista D 4.100.000, Lista B 1.500.000



COMPITI D'ESAME

Flight tracker, CupidOnline

Persistenza (package flightTracker.persistence)

(punti 11)

Sono forniti svariati file “.csv” nel formato fornito da FlightRadar24: il nome del file incapsula l’identificativo del volo e la data di effettuazione, nel formato *FlightID_AAAAMMGG* (ad esempio “*AZ604_20190510.csv*”). La prima riga contiene l’intestazione delle cinque colonne, separate da punti e virgola, mentre le successive contengono ciascuna un rilevazione, i cui campi sono anch’essi separati da punti e virgola. Da notare però che latitudine e longitudine costituiscono un unico campo, separato al suo interno da una virgola:

```
UTC;Position;Altitude;Speed;Direction
2019-05-10T10:54:39Z;45.661972,8.726303;1975;183;356
2019-05-10T10:54:49Z;45.67049,8.725822;2450;182;358
2019-05-10T10:55:11Z;45.689159,8.726234;3100;185;1
...
```

ESEMPIO DEL FILE Iscritti.txt

```
Roberto, M, 1994-04-24, capelli neri, occhi castani, 1.78, 61, Bologna, BO, Emilia-Romagna
Armando, M, 1997-08-01, capelli castani, occhi castani, 1.71, 65, Parma, PR, Emilia-Romagna
Eufrazio, M, 1993-10-30, capelli biondi, occhi azzurri, 1.82, 66, Firenze, FI, Toscana
Anna, F, 1996-06-18, capelli biondi, occhi azzurri, 1.70, 60, Imola, BO, Emilia-Romagna
Ludovica, F, 1992-10-14, capelli neri, occhi castani, 1.75, 51, Bologna, BO, Emilia-Romagna
Elena, F, 1999-04-10, capelli neri, occhi neri, 1.65, 57, Modena, MO, Emilia-Romagna
```

ESEMPIO DEL FILE Preferenze.txt

```
Roberto, F, 20-25, Bilancia, capelli biondi, occhi -, 1.70, 58, Bologna, BO, Emilia-Romagna
Armando, F, 17-25, -, capelli -, occhi -, -, -, Parma, PR, Emilia-Romagna
Anna, M, 20-29, Gemelli, capelli biondi, occhi azzurri, 1.70, 58, Imola, BO, Emilia-Romagna
Ludovica, M, 24-35, -, capelli -, occhi -, 1.80, -, Bologna, BO, Emilia-Romagna
Elena, M, 18-23, -, capelli neri, occhi azzurri, 1.75, 58, Modena, MO, Emilia-Romagna
```